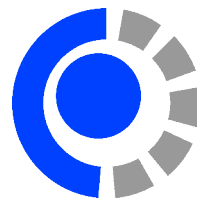




UNIVERSIDAD NACIONAL DEL COMAHUE
FACULTAD DE INFORMÁTICA



TRABAJO DE LABORATORIO N°3

DISEÑO DE COMPILADORES E INTÉRPRETES

Grupo 5

Franco Fabris

franco.fabris@est.fi.uncoma.edu.ar

FAI-3206

Valentina Villarroel

valentina.villarroel@est.fi.uncoma.edu.ar

FAI-3944

NEUQUÉN

ARGENTINA

2026

Contents

1	Descripción del problema	3
2	Estrategias utilizadas	3
3	Limitaciones	3
4	Resolución del enunciado	4
4.1	Alfabeto de entrada	4
4.2	Lexemas y componentes léxicos (Tokens)	4
4.3	Expresiones Regulares	4
4.4	Autómata Finito	5
5	Producto o Aplicativo	5
5.1	Descripción del producto y herramientas utilizadas	5
5.2	Manual de uso	5
5.3	Tabla de Errores y Tratamiento Sugerido	5
6	Problemas encontrados	7
7	Conclusión	7

1 Descripción del problema

El presente trabajo de laboratorio consiste en el diseño de un analizador léxico para un subconjunto del lenguaje Pascal. La tarea principal del analizador léxico es leer el flujo de caracteres del programa fuente, agruparlos en unidades con significado propio denominadas lexemas y producir una secuencia de tokens que serán utilizados por el analizador sintáctico.

Específicamente, se requiere:

- Definir el alfabeto de entrada y los componentes léxicos.
- Describir los patrones de dichos tokens mediante expresiones regulares.
- Diseñar un autómata finito capaz de reconocer estos componentes, incluyendo el manejo de comentarios encerrados entre llaves (`{ }`) y la omisión de espacios en blanco.

2 Estrategias utilizadas

Para el desarrollo del analizador léxico se aplicaron las siguientes estrategias:

1. **Clasificación de componentes:** Se categorizaron los elementos del lenguaje en grupos definidos (palabras reservadas, identificadores, números, operadores y puntuación) para facilitar su identificación en el flujo de caracteres.
2. **Implementación de un AF:** Se diseñó un **Autómata Finito** donde cada estado representa una etapa en el reconocimiento de un token. Este modelo garantiza que el análisis sea eficiente y que cada secuencia de caracteres sea asignada a una única categoría.
3. **Resolución de ambigüedades:** En el diseño del autómata se implementaron nodos de decisión para manejar caracteres que pueden formar parte de más de un token. Siguiendo el principio del *prefijo más largo*, el autómata evalúa el siguiente carácter antes de confirmar un símbolo, permitiendo distinguir correctamente, por ejemplo, entre un signo de menor (`<`) y un operador de distinto (`<>`).
4. **Gestión de elementos descartables:** Se definieron estados específicos para el tratamiento de espacios en blanco y comentarios entre llaves (`{ }`). Estos permiten que el analizador ignore información irrelevante para el análisis sintáctico sin interrumpir el proceso de reconocimiento.

3 Limitaciones

El diseño actual presenta las siguientes restricciones:

- **No anidamiento de comentarios:** El autómata reconoce comentarios simples `{...}`. Si se encuentra una llave de apertura dentro de un comentario, esta se trata como texto plano hasta hallar la primera llave de cierre.
- **Tipos de Datos:** Solo se contemplan números enteros y valores booleanos, excluyendo números reales o notación científica.

4 Resolución del enunciado

4.1 Alfabeto de entrada

Nuestro alfabeto de entrada Σ está compuesto por:

- **Letras:** a hasta la z. A hasta la Z.
- **Dígitos:** 0, 1, 2, 3, 4, 5, 6, 7, 8, 9.
- **Símbolos especiales:** +, -, *, /, =, <, >, :, ;, ., ,, (,).
- **Delimitadores de comentarios:** {, }

4.2 Lexemas y componentes léxicos (Tokens)

Los componentes léxicos (tokens) son las categorías abstractas generadas, mientras que los lexemas son las secuencias exactas de caracteres. A continuación, los definimos en la Tabla 2.

4.3 Expresiones Regulares

Utilizamos la notación de expresiones regulares para definir los patrones de los tokens identificados:

- $letra \rightarrow a \mid b \mid c \mid \dots \mid z \mid A \mid B \mid C \mid \dots \mid Z$
- $digito \rightarrow 0 \mid 1 \mid 2 \mid \dots \mid 9$
- $id \rightarrow letra(letra \mid digito)^*$
- $num \rightarrow digito^+$
- $op_rel \rightarrow = \mid < \mid > \mid < \mid < = \mid > \mid > =$
- $op_asig \rightarrow :=$
- $op_arit \rightarrow + \mid - \mid * \mid /$
- $punto_coma \rightarrow ;$
- $punto \rightarrow .$
- $dos_puntos \rightarrow :$
- $coma \rightarrow ,$
- $par_abre \rightarrow ($
- $par_cierra \rightarrow)$
- $cor_abre \rightarrow [$
- $cor_cierra \rightarrow]$

- *espacio* $\rightarrow (\text{blanco} \mid \text{tab} \mid \text{newline})^+$
- *comentario* $\rightarrow \{(\Sigma - \{\}\})^*\}$
- *palabra_reservada* $\rightarrow \text{program} \mid \text{var} \mid \text{integer} \mid \text{boolean} \mid \text{function} \mid \text{procedure} \mid \text{begin} \mid \text{end} \mid \text{if} \mid \text{then} \mid \text{else} \mid \text{while} \mid \text{do} \mid \text{read} \mid \text{write} \mid \text{true} \mid \text{false} \mid \text{or} \mid \text{and} \mid \text{not}$

4.4 Autómata Finito

El Autómata realizado se puede ver en la Figura 1, o en el siguiente link: Autómata Finito. Se asume el uso de una tabla de mapeo para las palabras reservadas. De esta manera, si una cadena de caracteres coincide con una entrada en la tabla, se asignará el token respectivo. De lo contrario, se devolverá como un identificador genérico.

5 Producto o Aplicativo

5.1 Descripción del producto y herramientas utilizadas

El aplicativo es un Analizador Léxico desarrollado íntegramente en el lenguaje de programación *Python 3*. Se optó por este lenguaje dada su alta legibilidad y la facilidad para manipular cadenas de texto y archivos. El programa no requiere dependencias externas para su análisis central, apoyándose únicamente en las bibliotecas estándar de *Python*. Para la generación del archivo ejecutable multiplataforma, se utilizó la herramienta *PyInstaller*, empaquetando el entorno en un único archivo independiente.

5.2 Manual de uso

El analizador se opera mediante una interfaz de línea de comandos (CLI). Para ejecutarlo, el usuario debe situarse en el directorio donde se encuentra el programa compilado y el archivo de código fuente a analizar.

1. Abrir una terminal o consola de comandos.
2. Ejecutar la siguiente instrucción, reemplazando *archivo.pas* por el nombre de su archivo de prueba: `$ analizador archivo.pas`
3. El aplicativo analizará el archivo y mostrará los errores léxicos por pantalla indicando el número de línea.
4. Una vez finalizado, se generará un archivo llamado *resultado_tokens.txt* en el mismo directorio, conteniendo los tokens generados.

5.3 Tabla de Errores y Tratamiento Sugerido

El analizador cuenta con mecanismos básicos de recuperación ante errores, informando por consola la línea exacta donde se produjo la falla.

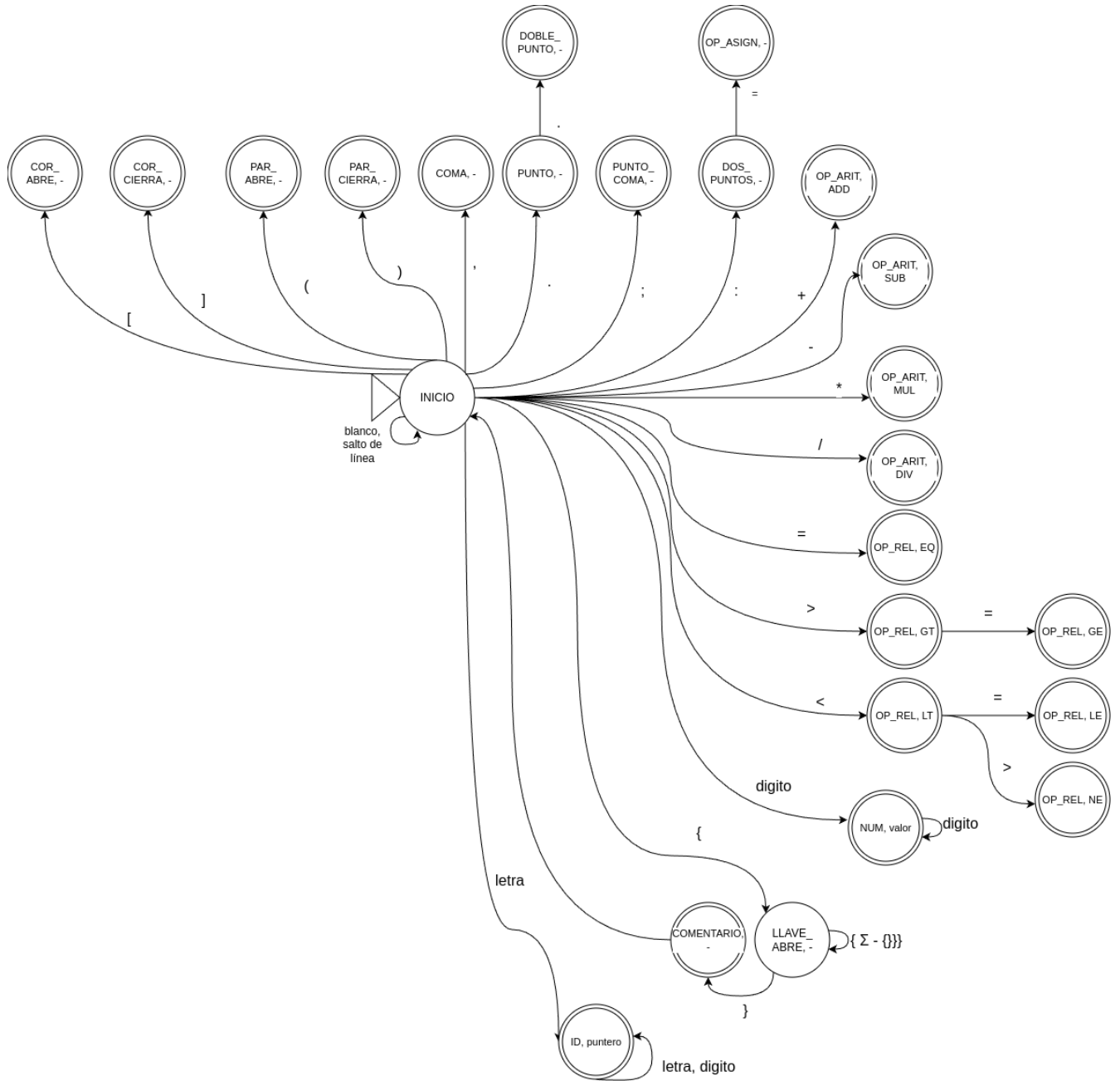


Figure 1: Autómata Finito

Mensaje Interno	Descripción	Fase de Detección	Tratamiento
UNRECOGNIZED_CHAR	Carácter fuera del alfabeto (ej: @, \$).	Lectura de caracteres.	Se ignora y avanza al siguiente carácter.
COMMENT	Comentario no cerrado (se alcanzó el EOF sin una }).	Lectura de comentarios.	Se reporta y finaliza la lectura del archivo.
NUM_ERROR	Número mal formado (ej: presencia de letras tras un dígito 123a).	Lectura de literales numéricos.	Se descarta el lexema completo y avanza.
ASIGN	Uso de : aislado (posible olvido de =).	Lectura de operadores.	Se emite un token de dos puntos genérico.

Table 1: Tabla de Errores y Mecanismos de Recuperación

6 Problemas encontrados

Uno de los mayores desafíos fue el manejo de los operadores relacionales y de asignación que comparten prefijos (como < y <=). Se resolvió mediante la implementación de estados intermedios que requieren mirar un carácter adelante para decidir la transición. Además, la correcta definición de la expresión regular para los comentarios requirió especial atención para asegurar que el autómata no se detuviera prematuramente ante caracteres especiales dentro de las llaves.

7 Conclusión

A través de este laboratorio, se logró comprender la importancia de la fase de análisis léxico como la base del compilador. La transición de expresiones regulares a un Autómata Finito permite automatizar el reconocimiento de patrones de forma eficiente. Se verificó que el uso de herramientas formales reduce drásticamente los errores en la detección de lexemas en comparación con métodos manuales.

Componente Léxico (Token)	Lexemas de Ejemplo	Valor de Atributo
Palabras Reservadas		
PROGRAM	program	-
VAR	var	-
INTEGER	integer	-
BOOLEAN	boolean	-
FUNCTION	function	-
PROCEDURE	procedure	-
BEGIN	begin	-
END	end	-
IF	if	-
THEN	then	-
ELSE	else	-
WHILE	while	-
DO	do	-
READ	read	-
WRITE	write	-
TRUE	true	-
FALSE	false	-
OR	or	-
AND	and	-
NOT	not	-
Identificadores y Números		
ID	x, suma, valor1	Puntero a la Tabla de Símbolos
NUM	0, 45, 999	Valor numérico entero
Operadores		
OP_ASIG	:=	-
OP_REL	=, <>, <, <=, >, >=	EQ, NE, LT, LE, GT, GE
OP_ARIT	+, -, *, /	ADD, SUB, MUL, DIV
Símbolos Especiales (Puntuación)		
PUNTO_COMA	;	-
PUNTO	.	-
DOS_PUNTOS	:	-
COMA	,	-
PAR_ABRE	(	-
PAR_CIERRA	)	-
Elementos Descartados		

Componente Léxico (Token)	Lexemas de Ejemplo	Valor de Atributo
Cualquier blanco / salto de línea	–	–
Cualquier comentario	{...}	–

Table 2: Tabla de componentes léxicos y lexemas